

Documento Explicativo — Solución de Validación de Número de Uniforme

****Quito Highlanders FC****

1. Problema Identificado

Actualmente, el proceso de registro de uniformes se realiza mediante un formulario de Google Forms. Cuando un representante ingresa el número de camiseta, ****no existe una validación automática**** que verifique si ese número ya está ocupado. La validación se hace manualmente contra una base de datos, lo que genera:

- ****Errores humanos****: se registran números duplicados.
- ****Retrabajo****: hay que contactar al representante, corregir el pedido y reprocesar.
- ****Fricción operativa****: el departamento de talento humano pierde tiempo validando manualmente.
- ****Mala experiencia****: el representante se entera después de que el número no estaba disponible.

****Regla de negocio****: la combinación `Número de camiseta` + `Categoría` + `Sede` debe ser única. Un número puede repetirse si cambia la categoría o la sede.

2. Lógica de la Solución

La solución implementa una ****validación en tiempo real**** en el frontend que verifica la disponibilidad mientras el usuario escribe o selecciona las opciones.

Flujo de validación

1. ****Usuario ingresa**** Categoría, Sede y Número.
2. ****¿Están los tres campos completos?****
 - No -> Mensaje de advertencia o estado neutro.
 - Sí -> Se dispara la función de validación (simulando latencia de red).
3. ****Consulta a la Base de Datos (Local Storage / Datos Semilla):****
 - Verifica si existe un jugador activo con el mismo número, categoría y sede.
4. ****¿Encontró coincidencia?****
 - Sí -> "Número no disponible — lo tiene: [Nombre]" (Bloquea el botón de envío).
 - No -> "Número disponible" (Permite enviar el pedido).

Validaciones adicionales

- ****Campos obligatorios****: evaluados visualmente y al intentar enviar.

- ****Feedback asíncrono****: muestra un spinner simulando la consulta.
- ****Prevención de envío****: el botón de envío lanza notificaciones (Toasts) si hay campos faltantes, evitando el envío silencioso.

3. Arquitectura Elegida

Componentes del sistema

- ****Aplicación Web (SPA) / React JS (Vite)****: Gestión del estado complejo, renderizado dinámico, validaciones y UX fluida.
- ****Base de Datos Dinámica / Local Storage API****: Persiste en tiempo real los registros guardados en el navegador para pruebas locales rápidas sin depender de un servidor externo (Serverless).
- ****Datos Base (Semilla) / Archivo JSON****: Archivo estático inicial que carga más de 60 registros en el navegador del evaluador en la primera visita.
- ****Notificaciones / API WhatsApp (Externa)****: Envía automáticamente vía método POST los resúmenes de pedidos al número introducido, enlazando la lógica con el endpoint `whatsappnotif.easysplus.com`.

Diagrama de la Arquitectura

`` `text

NAVEGADOR (Cliente React - 100% Frontend Serverless)

FormPage (Validación y UX) <----> useJugadores (Custom Hook Asíncrono)

| (POST WhatsApp) | (Guarda y Lee Data)

easysplus.com (WhatsApp) Local Storage (Persistencia Local)

^ (Carga inicial)

jugadores.json (Datos Base)

4. ¿Por qué esta arquitectura?

Alternativas consideradas

- **Google Forms nativo**: Fácil implementación, pero no permite validación en tiempo real contra una BD. (Descartada).
- **Google Apps Script (Web App)**: Gratis y se conecta directo a Sheets, pero el desarrollo de la Interfaz (UI) es rígido y la UX menos fluida. (Descartada).
- **React JS (Elegida)**: Ofrece UX Premium, máxima flexibilidad, escalable a cualquier backend futuro y permite un despliegue gratuito sin fricción.

Razones de la elección

1. **Experiencia de Usuario (UX)**: React permite validaciones ultra rápidas, estados de carga elegantes y un diseño profesional "Premium" que Google Forms no ofrece.
2. **Arquitectura Escalable Serverless**: Al usar Local Storage, el prototipo es 100% estático y se subió fácilmente a Vercel, permitiendo al evaluador probar la persistencia de datos instantánea sin depender de instalar un servidor de base de datos pesado.
3. **Automatización Inmediata**: Al completarse la orden, no solo se guarda la data, sino que automáticamente lanza una alerta POST a la API externa de WhatsApp completando todo el flujo logístico de manera programática.

5. Tiempos y Desarrollo

- **Análisis del problema y reglas**: ~30 min
- **Diseño UI / UX y arquitectura**: ~45 min
- **Desarrollo Frontend React (Componentes)**: ~2 h
- **Migración a JSON, LocalStorage y API WhatsApp**: ~1.5 h
- **Documentación y despliegue (Vercel)**: ~1 h
- **Tiempo Total Estimado**: ~5 horas y 45 minutos.

6. Limitaciones Actuales

- **Persistencia Local**: Al ser un entorno de prueba en Vercel sin servidor SQL, se usa `localStorage` para simular la base de datos. Los datos guardados solo existen en el navegador del usuario que hizo el registro.

- ****Volatilidad de Datos****: Si el usuario borra la caché de su navegador o usa modo incógnito, su "base de datos de pruebas" se reiniciará a los datos semilla.

7. Mejoras Futuras

- ****Integración a Backend Real****: Conectar React a una API oficial (ej. un backend con Laravel y base MySQL) o un endpoint Web App de Google Apps Script para persistencia global sincronizada.

- ****Autenticación (JWT)****: Crear un dashboard con Login cerrado para que el departamento de Talento Humano apruebe o rechace pedidos.

8. Conclusión

La solución implementada mediante ****React JS**** demuestra cómo elevar radicalmente la experiencia del usuario superando las severas limitaciones técnicas de los formularios tradicionales. La validación dinámica en tiempo real garantiza que la regla de negocio (Número + Categoría + Sede = Único) se cumpla estrictamente **antes** de enviar la información, eliminando la carga de retrabajo operativo y dotando al club de una solución moderna, programática, escalable y muy eficiente.